

Projet1AB Helios

Architecture du code

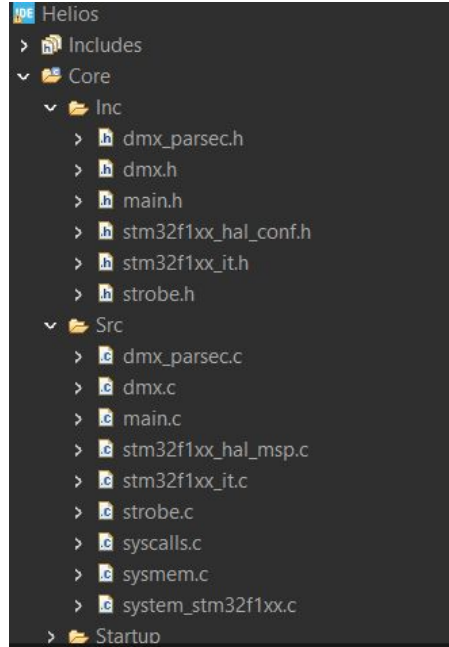


Table des matières

1. Architecture globale
2. DMX512 : explication
3. Le module DMX
4. Le module DMX_PARSE
5. Le module STROBE
6. Difficultés rencontrées et apprentissages
7. Bibliographie



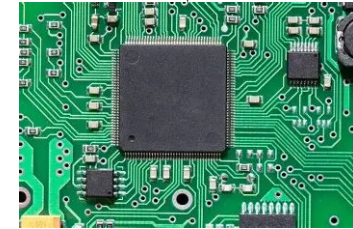
1. Architecture globale



DMX512 +
RS485

Driver

Code:
Dmx.c(réception)
Dmx_parsec.c(Analyse)
Strobe (PWM emission)



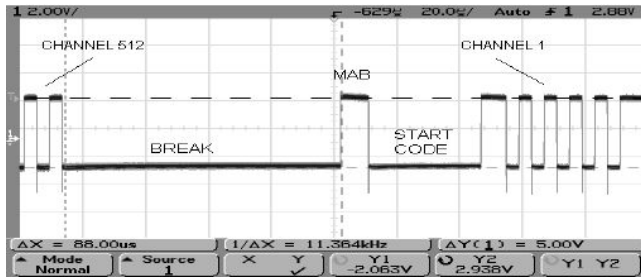
USART1 STM32



LED

2. DMX512 : Explication

La norme DMX512 fonctionne par envoi d'octets pour chaque channel (donc de 0 à 255 codé sur 8 bits).



Cela va nous permettre d'établir l'intensité de chaque channel avec une PWM sur notre driver, selon la formule :

intensité = (valeur du channel × intensité du master) / 255

La norme DMX512 est un protocole d'éclairage numérique utilisé dans le domaine du spectacle et de la scène. Elle repose sur une communication série RS-485. Le protocole DMX512 permet de transmettre jusqu'à 512 canaux et fonctionne selon une architecture maître → esclaves : une console maître peut commander plusieurs équipements esclaves.

Nous utilisons le RS-485 et non le RS-422 car le RS-485 est plus adapté aux systèmes DMX. Bien que les deux utilisent une transmission différentielle, le RS-485 permet d'avoir plusieurs équipements sur le même bus, contrairement au RS-422 qui ne supporte qu'un seul émetteur.

Ainsi, plusieurs appareils peuvent être connectés sur la même ligne : projecteurs, strobes, machines à fumée, etc.

Le RS-485 possède également une forte immunité aux parasites électriques et est adapté aux communications sur de longues distances.

3. Le module DMX

```
7
8 #ifndef INC_DMX_H_
9 #define INC_DMX_H_
10
11 #include "stdint.h"
12 #include "main.h"
13
14 #define DMX_CHANNELS 512
15
16 void DMX_Init(void);
17 void DMX_StartReception(void);
18
19 // Getter
20 uint8_t DMX_GetChannel(uint16_t channel);
21
22 // Status
23 uint8_t DMX_NewFrameAvailable(void);
24 void DMX_ClearFrameFlag(void);
25 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart);
26
27 void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart);
28
29 #endif /* INC_DMX_H_ */
30
```

Nombre de channel DMX

Initialisation de la réception de données DMX

Getter de la donnée DMX

Etat de la donnée (si c'est une nouvelle émission
et si l'émission précédente a été reçue)
(Réception de l'interruption)

Ce module s'occupe de la réception de la
donnée DMX

4. Le module DMX_PARSEC

```
7
8 #ifndef INC_DMX_PARSEC_H_
9 #define INC_DMX_PARSEC_H_
10
11 #include "stdint.h"
12 #include "main.h"
13
14 #define DMX_CH_STROBE_ENABLE    0
15 #define DMX_CH_STROBE_SPEED    1
16 #define DMX_CH_RED              2
17 #define DMX_CH_GREEN           3
18 #define DMX_CH_BLUE            4
19
20 #define OCTET 255
21
22 typedef struct{
23     uint8_t strobe_enable;
24     uint8_t strobe_speed;
25     uint8_t red_intensity;
26     uint8_t green_intensity;
27     uint8_t blue_intensity;
28 } DMX_ParsedData_t;
29
30 void DMX_ParseFrame(void);
31 DMX_ParsedData_t* DMX_GetParsedData(void);
32
33 #endif /* INC_DMX_PARSEC_H_ */
```

Structure des données

Analyse des données

Ce module analyse
les données et met à
jour les données pour
les strobes



5. Le module STROBE

```
7 #ifndef STROBE_H
8 #define STROBE_H
9
10 #include "stdint.h"
11 #include "dmx_parsec.h"
12 #include "stm32f1xx_hal_tim.h"
13
14 #define ON 1
15 #define OFF 0
16 #define START 0
17 void STROBE_Init(void);
18 void STROBE_Update(DMX_ParsedData_t* data);
19 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim);
20
21 #endif /* INC_STROBE_H */
```

Initialise les GPIO
et le timer pour les
PWM

Met à jour les
donnée pour les
LEDs

Ce module convertit les
données logiques en
physique.

Permet de configurer le
clignotement des leds

6. Difficultés rencontrées et apprentissages

Difficultés rencontrées :

- Manque de temps pour tester le bon fonctionnement du code
- Surchauffe du PCB empêchant certains tests .
- Erreur lors du routage sur l'association des pins :
Driver de LEDG2 et LEDB2 ont été connectés à des pins ne disposant pas de la fonction TIMER (Solution mise en place :
Création d'une PWM software pour piloter les LEDs concernées)

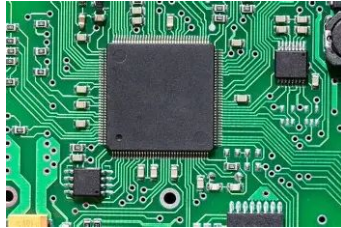
Apprentissages :

- Utilisation de l'oscilloscope pour visualiser l'émission et la réception des trames DMX
- Rédaction d'un code plus propre et modulaire avec l'utilisation de getter, setter et machine à état.
- L'importance du choix et du branchement des pins lors du routage et de la conception du schématique.

7. Bibliographie



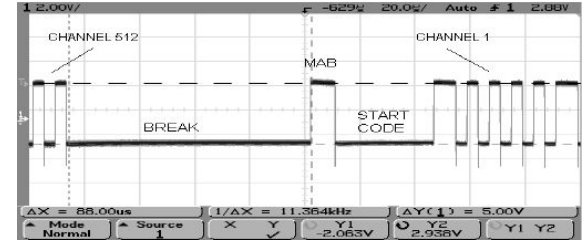
[DMX console rental for 6 channels | FX machines](#)



[Conception de PCB de microcontrôleur : un guide complet](#)



[François-Xavier Pourre – Light ZOOM Lumière – Portail de la Lumière et de l'Éclairage](#)



[Complete DMX512 Signal. | Download Scientific Diagram](#)
Par Christos Chousidis